

- ☐ No syntax highlighting
- ☐ Hide deleted text

try_*_back For Other Sequence Containers

Document #: P4228R0 [[Latest](#)] [[Status](#)]
Date: 2026-05-12
Project: Programming Language C++
Audience: LEWG
Reply-to: Nevin “:-)” Liber
[<nliber@anl.gov>](mailto:nliber@anl.gov)

Contents

- [1 Revision History](#)
 - [1.1 R0](#)
- [2 Introduction](#)
- [3 Motivation and Scope](#)
- [4 Impact On the Standard](#)
- [5 Design Decisions](#)
 - [5.1 Add try_*_back to vector](#)
 - [5.2 Do not add try_*_back to deque](#)
 - [5.3 Do not add try_*_back to list](#)
 - [5.4 Do not add unchecked_*_back to vector, deque or list](#)
- [6 Technical Specifications](#)
 - [6.1 Add to \[vector.overview\]:](#)
 - [6.2 Add to \[vector.modifiers\]:](#)
 - [6.3 Add to \[vector.bool.pspc\]:](#)
 - [6.4 Add to \[version.syn\]:](#)
- [7 Acknowledgements](#)
- [8 References](#)

§ 1 Revision History

§ 1.1 R0

First proposed.

§ 2 Introduction

Now that we have `try*_back` in `inplace_vector`, we should add it to `vector` and explore adding it to other sequence containers that provide `push_back` and `emplace_back`.

§ 3 Motivation and Scope

In P0843R7, `try_push_back` and `try_emplace_back` were added to `inplace_vector`. They are just as useful in `vector`.

For example: when working on a low latency system, it is common to pre-allocate the maximum capacity needed in a vector, then on the “hot path” (low latency path) to just `push_back` elements, knowing that the vector will not internally reallocate and iterators, pointers and references to elements will not be invalidated.

§ 4 Impact On the Standard

This is purely additive.

§ 5 Design Decisions

§ 5.1 Add `try*_back` to vector

The salient difference between `inplace_vector` and `vector` in this case is whether or not the capacity is known at compile time or only at run time. There are, of course, other differences (such as whether or not one wants the elements embedded in the object or on the heap) that may affect which container to use, but in either case, they should be able to use the same API.

This paper proposes adding `try_push_back` and `try_emplace_back` to `vector` and `vector<bool>`. They would have the same exact semantics as they do for `inplace_vector`, including the same specification.

§ 5.2 Do not add `try*_back` to deque

One could generalize this even more to `deque` so that it returns an optional `<T&>` that is engaged if and only if the deque did not perform an internal allocation (either another

chunk block or resizing the map of pointers). However, there is currently no method to set the capacity, so it is not quite as useful here.

If we were to eventually add it, we should also add the corresponding `try*_front` functions.

It is the author's recommendation that we do not add these functions to `deque`.

§ 5.3 Do not add `try*_back` to `list`

If both `vector` and `deque` were to get this, `list` should as well, even though it would always return a disengaged `optional<T&>`.

If we were to eventually add it, we should also add the corresponding `try*_front` functions.

It is the author's recommendation that we do not add these functions to `list`.

§ 5.4 Do not add `unchecked*_back` to `vector`, `deque` or `list`

While these functions were added to `inplace_vector` in order to gain consensus on the entire proposal, the tradeoff of adding an API with an obvious memory safety issue for a questionable micro-optimization (avoiding the internal `if` check against the capacity) isn't worth doing. It is also unclear if these unchecked functions will really remain unchecked under a memory safety profile. Should someone wishes to propose this, they need to provide benchmarks showing the benefits.

It is the author's recommendation that we do not add these functions to `vector`, `deque` or `list`.

§ 6 Technical Specifications

These are relative to N5032:

§ 6.1 Add to `[vector.overview]`:

```
// modifiers
template<class... Args> constexpr reference emplace_back(Args&&... args);
constexpr void push_back(const T& x);
constexpr void push_back(T&& x);
template<container-compatible-range <T> R>
    constexpr void append_range(R&& rg);
constexpr void pop_back();

template<class... Args>
constexpr optional try_emplace_back(Args&&... args);
```

```
constexpr optional try_push_back(const T& x);  
constexpr optional try_push_back(T&& x);
```

§ 6.2 Add to [vector.modifiers]:

```
template<class... Args>  
    constexpr optional<reference> try_emplace_back(Args&&... args);  
constexpr optional<reference> try_push_back(const T& x);  
constexpr optional<reference> try_push_back(T&& x);
```

Let vals denote a pack:

- `std::forward<Args>(args)...` for the first overload,
- `x` for the second overload,
- `std::move(x)` for the third overload.

Preconditions: `value_type` is *Cpp17EmplaceConstructible* into vector from vals....

Effects: If `size() < capacity()` is true, appends an object of type `T` direct-non-list-initialized with vals....

Otherwise, there are no effects.

Returns: `nullopt` if `size() == capacity()` is true, otherwise `optional<reference>` (`in_place`, `back()`).

Throws: Nothing unless an exception is thrown by the initialization of the inserted element.

Complexity: Constant.

Remarks: If an exception is thrown, there are no effects on `*this`.

§ 6.3 Add to [vector.bool.pspc]:

```
// modifiers  
template<class... Args> constexpr reference emplace_back(Args&&... args);  
constexpr void push_back(const bool& x);  
template<container-compatible-range <bool> R>  
    constexpr void append_range(R&& rg);  
constexpr void pop_back();  
  
template<class... Args>  
constexpr optional try_emplace_back(Args&&... args);  
constexpr optional try_push_back(const T& x);  
constexpr optional try_push_back(T&& x);
```

§ 6.4 Add to [version.syn]:

```
#define __cpp_lib_vector [YYYYMML] // also in <vector>
```

§ 7 Acknowledgements

I thank all the WG21 members who have litigated this since I started this quest in 2015. It has made for better proposals, and hopefully made me a stronger debater.

§ 8 References

- [N5032](#) Working Draft, Programming Languages - C++
- [N4416](#) Don't Move: vector Can Have Your Non-Moveable Types Covered
- [P0843](#) `inplace_vector`
- [P3830](#) NB-Commenting is Not a Vehicle for Redesigning `inplace_vector`
- [P3981](#) Better return types in `std::inplace_vector` and `std::exception_ptr_cast`